ChatGPT

我现在用的工具是codex openclaw； codex及vs codex中开发 openclaw插件，我希望能做到 自我反馈自我修复的系统：
1. openclaw 一个code channel 告知插件 我要什么 ， 插件去开发，部署，
2. openclaw一个chanal 使用插件，
3. 插件能够自动检查log，发现不符合预期，就自动分析问题，汇报问题（发到code channel，说问题，讨论解决思路），然后解决问题，部署；

最终形成 用户使用-》自动发现问题-〉自动解决问题-》自动部署-〉用户继续使用 这样一个循环；
当然，code channel 也可以提出新的需求， 经过讨论后，创建一个新的项目，然后进入使用-》自动完善-〉使用；

背景：我写了这个插件，使用时明显有问题，我要让codex去查问题，解决问题，部署，我自己在使用，在发现问题，在告诉codex， 这不是很笨的做法么？ 既然已经在前期和使用过程中 告知了codex 插件是什么样的， 有什么功能，有什么问题， 为什么不是它自己发现问题，自己解决问题呢？

已思考 ›

已思考 ›

已思考 ›

已思考 ›

已思考 2m 31s ›

你这个想法不笨，反而是对的。

你真正想要的，不是“让 Codex 帮你写代码”，而是做一个**闭环的软件代理系统**：会观测、会判断、会修、会发版、会继续观测。OpenAI 现在把 Codex 定位成 coding agent：它能

读写运行代码，云端任务也能后台并行执行；Codex app 还有 automations、worktrees、skills 这些机制。但官方文档也写得很清楚：Codex 最适合被当成一个**有明确上下文、明确 definition of done 的队友**，而不是天然就会自我治理的常驻生产系统。它的动作也受到 sandbox、approval、network 权限约束。 OpenAI 开发者 +3

而 OpenClaw 本身更像你的**控制平面**：它是一个自托管 gateway，负责把不同 channel 接进来；plugin 则负责扩展 channel、tool、skill 等能力。也就是说，你的方向本质上是：**用 OpenClaw 做“事件与渠道总线”，用 Codex 做“分析与修复执行器”**。这个方向和 OpenClaw 的架构是吻合的。 GitHub +1

你现在觉得“我自己用、自己发现 bug、再告诉 Codex，很笨”，核心原因不是 Codex 不够聪明，而是少了中间那一层：**自动反馈基础设施**。

为什么它不能天然“自己发现、自己解决”？

1. **知道功能，不等于知道哪里算错。**
   你前期告诉过 Codex“这个插件应该长什么样”，这只是自然语言理解。要让它自动发现问题，必须把“预期”变成机器可判定的东西：测试、断言、SLO、日志模式、验收脚本、回归用例。官方也建议把难题做成“scored improvement loop”，给 Codex 一套 eval system、脚本和可审阅产物，让它按分数迭代，而不是靠模糊印象。
   OpenAI 开发者 +2
2. **知道历史上下文，不等于持续看到现场。**
   一次对话里说过“这个插件是干什么的”，不代表它之后持续看到真实用户请求、运行日志、异常堆栈、延迟、失败率、部署结果。没有这些实时信号，它就没有“发现问题”的眼睛。Codex 的自动化和云端任务都依赖你给它环境、命令、上下文和可执行检查。云端环境里，Codex会跑命令、改代码、跑检查、尝试验证，但前提是这些检查本来就存在。 OpenAI 开发者 +2
3. **会修 bug，不等于应该直接上线。**
   真实系统里，自动修复涉及权限、网络、secret、部署边界、回滚边界。Codex 默认就不是“无限权力模式”：官方文档明确说 agent 默认网络是关的，很多动作受 approval policy 和 sandbox 限制。这个不是缺点，而是为了避免“模型一激动把生产环境也改了”。 OpenAI 开发者 +1
4. **一次会话记住了，不等于形成长期运行的状态机。**
   你想要的是一个常驻 loop：
   观察 → 判断异常 → 复现 → 修复 → 验证 → 发布 → 再观察。
   这不是“多给一点上下文”就会出现的能力，而是要外部系统持续触发。Codex app 的 automations 能做后台重复任务，但它也有边界：app 需要开着，项目要在磁盘上；云端任务能后台并行，但依然是你配置好的任务，而不是全知全能的自治守护进程。
   OpenAI 开发者 +2

所以，正确的理解应该是：

Codex 不是那个闭环本身。Codex 是闭环里的“高智能工人”。
真正的闭环，要靠你把“观察、判定、升级、修复、部署、回滚”这整套外壳搭出来。

我建议你把系统拆成四层：

第一层：使用层
一个 user/use channel，真实用户就在这里使用插件。

第二层：观测层
把日志、报错、关键事件、用户失败案例、健康检查结果，统一结构化。不是一坨 log，而是事件对象，比如：

- 触发时间
- 用户动作
- 预期结果
- 实际结果
- 相关日志片段
- 版本号/commit
- 可复现步骤

第三层：判定层
专门有一套 detector/eval：

- smoke test
- 合成用户流程
- 关键错误日志模式匹配
- 行为偏差检查
- LLM 归因分析

这里最关键。因为“自动发现问题”本质上不是靠 Codex 猜，而是靠**判定器先把异常框出来**。

第四层：执行层
Codex 来做：

- 读 incident
- 分析根因
- 在 worktree 里修
- 跑测试
- 生成变更说明
- 部署到 staging
- 观察结果
- 满足门槛后再部署

这一步，Codex 很强。官方也强调 skills 要聚焦、输入输出要明确；automations 可以和 skills 组合。也就是说，你要把“查日志→复现→修复→验证→报告”封成一组稳定 workflow，而不是每次临场口头说。 OpenAI 开发者 +1

你想要的 code channel / use channel 设计，我建议这样落：

- **use channel**：只服务用户，不讨论开发。
- **code channel**：只接收三类内容：新需求、事故报告、自动修复报告。
- **observer/ops 通道**：不一定给人看，专门接 log anomaly、smoke 失败、部署结果。
- **deploy 通道**：记录当前版本、回滚点、发布状态。

这样，“用户说有问题”和“系统自己发现问题”最后都会落到同一种结构化 incident 上，Codex 才能稳定处理，而不是一会儿靠聊天理解、一会儿靠你补充说明。

更重要的是，你现在应该采用的不是“一步到位全自动”，而是三阶段：

阶段 1：半自动闭环
用户使用 → 系统收集异常 → 自动发到 code channel → Codex 分析并给修复方案 → 你批准部署。
这是最现实的起点。

阶段 2：安全范围内自动修复
只对低风险问题自动修：

- 明确报错
- 可复现
- 有测试可验证
- 无 schema 变更
- 无权限/账单/安全影响

阶段 3：近似自治
只有当你的检测器、测试、回滚、部署门禁足够成熟时，才让它自动上线。否则“全自动”很容易变成“全自动制造新事故”。

一句话总结：

你不是该让 Codex “更懂你”，而是该把“用户反馈”升级成“机器可判定信号”，再把 Codex 接到这个信号回路里。
当回路搭起来后，你就不再是“自己发现 bug 再告诉 Codex”；而是“系统先发现，Codex先处理，你只在关键风险点介入”。

最值得坚持的一条工程原则是：

每一个你亲自报给 Codex 的问题，都要在修完后升格成一个自动检测项。

这样你报一次，它以后就少靠你一次。
报十次，系统就真的开始长出“自我反馈、自我修复”的雏形了。

下一条我直接给你一版可落地的方案：
OpenClaw channel 分工 + 仓库目录结构 + AGENTS.md 模板 + incident 数据结构 + Codex automation 提示词。

来源

codex云端是否要单独收费， 直接在我的电脑端是否能解决问题?
每个插件（现在是插件未来可能是个软件）要自己定一自己的功能和问题边界，并形成文档； 把这个文档丢和运行日志给codex或者llm，llm就能够知道是否有问题， 然后就可以调用codex写代码（甚至不用调用它，那调用谁呢？还是都不用，llm自己干？），然后自己检查，自己发布（目前是到本地），这个链路应该是清晰的；

已思考 ›

已思考 ›

已思考 2m 19s ›

可以分成两件事看。

1）Codex 云端是不是要单独收费?

不是“必须额外单买一个云端订阅”这种模式。现在 Codex 已经包含在 ChatGPT Plus、Pro、Business、Edu、Enterprise 这些计划里；同时，ChatGPT 计划内的包含用量会先消耗，超出后可以再买 credits 继续用。另一个分支是：如果你在 Codex CLI、IDE 扩展或 app 里用 **API key** 登录，而不是用 ChatGPT 账号登录，那就会走 OpenAI Platform 的标准 API 计费，不走 ChatGPT 计划里的 credits。Codex cloud 本身要求用 ChatGPT 登录；CLI 和 IDE 扩展两种登录方式都支持。 OpenAI 开发者 +2

对你这种场景，更准确的理解是：**云端不是单独“买不买”的问题，而是你用哪种运行面、走哪种计费路径。** 如果你是用 ChatGPT 计划登录 Codex，本身就有包含额度；如果你是把它当成程序化基础设施去跑，常见做法反而是 API key + SDK/CLI，这时按 API 计费。

OpenAI 开发者 +1

2）直接在你电脑端能不能解决？

能，而且对你现在这个 **OpenClaw 插件 + 本地部署 + 本地看日志** 的阶段，我更建议先用本地把闭环跑通。Codex CLI 官方定义就是：它能在你机器上的选定目录里读代码、改代码、跑命令；VS Code 扩展也可以在 IDE 里直接工作，或者把长任务委托给 Codex Cloud。Codex app 的 automations 还能直接跑在你的本地项目里，或者跑在独立 worktree 里，这两种都是后台运行。 OpenAI 开发者 +2

云端的主要价值，不是“更聪明”，而是**后台并行、独立隔离环境、你不占着本机也能继续跑长任务**。官方对 Codex cloud 的描述也是：它在自己的云环境里后台工作，而且可以并行。 OpenAI 开发者 +1

所以按你现在的目标，我会这样判断：
本地先做成，云端后接入。
因为你的真实运行日志、开发环境、部署目标现在都更贴近本机，本地闭环最短；等闭环稳定后，再把“长分析、并行修复、PR 生成、回归巡检”这类任务外包给 cloud。这个顺序更顺。

3）你说的那条链路，方向是对的，但少了一层“可判定标准”。

你说“每个插件定义自己的功能和问题边界，形成文档，再把文档和运行日志丢给 Codex/LLM，让它判断有没有问题、写代码、检查、发布”，这个思路是对的。
但**文档 + 日志，只够分析，不够稳定判定。**

官方 best practices 里其实讲得很直白：Codex 想要稳定，任务上下文里最好明确四类东西——**Goal**、**Context**、**Constraints**、**Done when**；同时把可复用规则放进 `AGENTS.md`，包括 repo 结构、怎么运行、build/test/lint 命令、约束、以及“完成算什么”。 OpenAI 开发者

也就是说，你不能只给它一份“产品说明书”，还要给它一份**机器可验收的合同**。至少要有这些：

- 功能边界文档：它应该做什么，不应该做什么。
- `AGENTS.md`：怎么启动、怎么测试、怎么验证、什么叫修好了。
- 日志规范：哪些日志字段是关键，哪些异常模式算 incident。
- smoke tests / integration tests：用户链路是否通过。
- 发布与回滚脚本：修完怎么发，本地失败如何撤回。

官方也明确建议，不要只让 Codex“改一下代码”，而要让它**创建/更新测试、跑相关检查、确认结果、审查 diff**。只有它知道“好”长什么样，那个自我修复循环才会稳。 OpenAI 开发者

4）“LLM 自己干”还是“调用 Codex”？

这个要分清楚：

- **裸 LLM**（例如直接用 `gpt-5.4` 的 API）更像“脑子”，适合分析日志、读文档、归纳问题、提出修复计划。
- **Codex** 更像“带手的工程代理”，官方定位就是现成的 coding agent，能读代码、改代码、跑命令、调试、接入 IDE / CLI / SDK / CI/CD。 OpenAI 开发者 +1

所以，真正能落地的做法通常不是“只靠 LLM 自己干”，而是：

LLM/Codex 负责思考与决策，Codex CLI / IDE / SDK 负责实际执行。
如果你要程序化接入，你可以直接用 **Codex SDK**。官方对 SDK 的定义就是：用来程序化控制本地 Codex agents，接入 CI/CD，构建你自己的 agent，或者嵌进内部工具。

OpenAI 开发者

换句话说：

- 你要“分析一个异常、总结原因、给出方案”，裸模型就能做。
- 你要“进 repo 改代码、跑测试、看结果、自动部署”，更合理的是走 Codex agent surface。
- 你要“让 OpenClaw 触发这个流程”，就用 OpenClaw 当编排层，去调 Codex SDK/CLI。

5）落到你这个 OpenClaw 场景，我建议先这样搭：

第一版不要追求“完全自治”，先做 **半自治闭环**：

1. **use channel**
   用户正常使用插件。
2. **observer**
   把运行日志、异常、失败用例、关键指标收集成结构化 incident。
3. **code channel**
   只接收三类输入：新需求、incident、修复报告。
4. **Codex 执行器**
   用本地 Codex CLI 或 Codex SDK 去：
   - 读取 incident
   - 对照 `AGENTS.md`
   - 复现问题
   - 改代码
   - 跑测试 / lint / smoke
   - 本地部署
   - 输出报告到 code channel
5. **自动化边界**
   只有满足“低风险 + 可验证 + 可回滚”时，才自动发版；否则只自动修到待审阶段。

这个路线和官方 best practices 很一致：先把可重复流程做成 **skills**，再把稳定流程做成 **automations**；适合 skills 的典型任务里就包括 log triage、incident summary、standard

debugging flows。 OpenAI 开发者 +2

我给你一句结论：

对你现在这件事，最优解不是先上 Codex cloud，而是先把“插件规格 + AGENTS.md + 测试/验收 + 结构化日志 + 本地 Codex 执行器”这五件事接起来。
这套先跑顺了，云端只是把它放大，不是替代它。

下一条我可以直接给你一版可落地骨架：
OpenClaw 三通道设计 + `AGENTS.md` 模板 + incident JSON 结构 + Codex 执行 prompt。

来源

先别急，我的问题在深入下，其实问题分两类：
1. llm 无法判断是不是问题， 需要人告诉他
2. 本地testcase能跑通，但是真实环境往往要复杂很多，那真是环境 插件的回复或者运行是否正确， llm自己完全能判断；

那这两类，第一类，人工可以给他反馈问题，告知希望状态； 然后自己去修； 第二类，自己就能改；

本来对人类来讲。可以更简单的反馈问题，ai自动修复；那为什么还要现在这么笨重的工作方式呢（我坐在电脑旁，和ai一起调试，其实无非就是给他更明确的指令，但是这些指令 ai可以自己分析，并做到的）？

Thought for 11m 4s

你这个追问，已经问到根上了。

真正的分界线，不是“AI会不会写代码”，而是：**谁在给它“判错信号”**。

OpenAI 自己给 Codex 的最佳实践，其实已经把这件事说透了：Codex 想稳定工作，必须知道 `Goal / Context / Constraints / Done when`，而且它能自己跑“改代码→跑检查→确认结果→复核”这个闭环，前提是它先知道“什么叫好”。这个“好”可以来自 prompt，也可以来自 `AGENTS.md`。 OpenAI 开发者 +2

所以你说的两类问题，我基本同意，但我会把它说得更精确一点。

第一类：规范缺失型问题。
这类问题不是“程序崩了”，而是“虽然能跑，但不符合我的真实期待”。比如回复口吻不对、流程设计别扭、业务边界踩错、优先级判断不符合你的产品直觉。对这类问题，LLM 不是完全不会修，而是**它不能凭空知道这算不算错**。除非你把偏好外化成文档、规则、验收标准、反例、rubric，或者放进 `AGENTS.md` 里。Codex 官方也明确建议把这种可复用指导沉淀到 `AGENTS.md`，而不是反复口头纠正；如果同样的错误出现第二次，就应该让 Codex 做 retrospection 并更新 `AGENTS.md`。 OpenAI 开发者 +1

第二类：运行可观测型问题。
这类问题就不是“你觉得不爽”，而是系统自己能看到异常：日志报错、输出格式错、接口超时、UI 渲染异常、e2e 流程失败、线上行为偏离预期。对这类问题，你的直觉也是对的：**很多时候 AI 确实可以自己发现、自己修。** Codex 官方就把 log triage、telemetry/incident summaries、standard debugging flows 列为很适合做成 skills 的任务；而且当一个流程足够稳定后，可以再做成 automations，后台定时跑。对于仓库外、而且经常变化的上下文，官方建议用 MCP 直接把外部工具和系统接给 Codex，而不是靠人手复制粘贴。

OpenAI 开发者 +3

但这里还要加半句：你说“真实环境里插件回复或者运行是否正确，LLM 自己完全能判断”，**这个“完全”通常还是太乐观了。**

因为“正确”其实分三层：

1. **运行正确**：没报错，状态码对，日志干净。
2. **行为正确**：真的完成了你想要的流程，不只是表面成功。
3. **业务正确**：这件事值不值得这样做，是否符合产品策略、风险偏好、用户体验。

第 1 层最容易自动化。
第 2 层要靠更好的 e2e、回放、artifact 检查。
第 3 层往往还需要人，或者至少需要你先把人的判断方式做成一个 judge。

OpenAI 在迭代难题的文档里给的办法，其实就是这个思路：把评估拆成**确定性检查**和 **LLM-as-a-judge** 两部分，再给它明确的停止规则。也就是说，主观的那部分不是不能做，而是要先被你“格式化”为可复用的 judge。 OpenAI 开发者 +1

所以，为什么你现在还会觉得流程“笨重”？

不是因为你真的在做多余工作，而是因为**你现在正亲自扮演系统里还没被产品化的四个模块**：

- 你在扮演 **判题器**：这算不算问题。
- 你在扮演 **现场观察器**：真实环境里哪里坏了。
- 你在扮演 **风险审批器**：这个改动能不能直接发。

- 你在扮演 **停止条件**：修到什么程度算够了。

而 Codex 默认并不会天然越过这些边界。官方文档写得很清楚：本地默认有 OS 级 sandbox，网络默认关闭，并且 approval policy 决定它什么时候必须停下来征求你同意；同时官方也特别提醒，很多质量问题其实是环境配置问题，所以要尽早把真实环境、工具、权限和上下文接对。 OpenAI 开发者 +2

所以你现在“坐在电脑旁边和 AI 一起调试”，本质上不是在给它补充聪明，而是在给它补四样东西：**目标、现场、权限、验收**。

这就引出一个很关键的结论：

你未来不是要继续做人肉 debugger，而是要把你现在的人肉判断，逐步变成机器可复用的 judge。

也就是把每一次人工介入，沉淀成下面四种资产之一：

- 一个 detector：什么日志模式/行为偏差算 incident
- 一个 eval：怎么判断修复有没有真的变好
- 一条 rule：哪些改动能自动发，哪些必须审批
- 一份 guidance：写进 `AGENTS.md` / skill / 任务模板里

这也正好符合 Codex 官方给出的路线：重复纠正的流程，先做成 skill；同样错误反复出现，就更新 `AGENTS.md`；流程稳定后，再做成 automation。 OpenAI 开发者 +2

所以，答案其实是：

你现在觉得笨重，不是因为这条路错了，而是因为你还处在“把人的隐性判断显式化”的启动阶段。
这个阶段确实会重。
但它不是最终形态。

最终形态不是“人一直坐旁边给 AI 下更明确的指令”，而是：

- 第一类问题：人类只负责定义偏好和边界，AI 负责按边界修。
- 第二类问题：系统自己发现异常，AI 自己分析、自修、自测、自部署。
- 每一次人类的新反馈，都只出现一次；以后就变成 detector / eval / rule，再也不用重复说。

一句话讲透：

你不是在教 AI 怎么修 bug；你是在教系统以后怎样不用再来问你，这个算不算 bug。

下一步最有价值的，不是继续聊抽象架构，而是把你最近碰到的 10 个真实问题，逐条分成“规范缺失型”还是“运行可观测型”，然后分别沉淀成 detector、eval、rule、或 `AGENTS.md` 条目。

来源

我最终希望做的是 自动反馈式的 软件编写； A窗口告知我要什么，软件自动实现，并部署； B窗口在用； A窗口反馈； 然后后台自动演化；

Thought for 10m 20s

对，这个目标是成立的。
而且它其实已经不是“AI 辅助编程”了，而是**自动反馈式的软件工厂**。

你要的不是一个会写代码的助手，而是一个持续运行的闭环：

```
A窗口（需求 / 反馈 / 裁判）
        ↓
    规格化与判题层
        ↓
    实现层（写代码 / 改代码）
        ↓
    验证层（测试 / 回放 / judge）
        ↓
    部署层（本地 / staging / 回滚）
        ↓
B窗口（真实使用）
        ↓
  观测层（日志 / 行为 / 异常 / 反馈）
        └──────────────→ 回到A或直接进入修复
```

你现在终于把事情说清楚了：
A窗口不是“让 AI 写代码”的聊天框。A窗口其实是产品控制台。
B窗口不是“给你看结果”的界面。B窗口其实是真实世界的证据源。
后台则不是一个裸 LLM，而是一套固定回路。

这套系统真正的核心

你最后要自动化的，不只是“写代码”，而是三件事：

第一条回路：造软件

A 窗口说“我要什么”
→ 系统把它变成规格、边界、验收条件
→ 写代码、测试、部署
→ B 窗口出现可用的软件

第二条回路：修软件

B 窗口真实使用
→ 日志、失败、异常行为被观测到
→ 系统自己判断这是 incident
→ 自动分析、修复、验证、部署

第三条回路：学软件

A 窗口再给反馈
→ 不是只改这一次代码
→ 而是把这次反馈沉淀成：

- 产品规则
- judge 规则
- 回归测试
- 不可违反约束

这一步最重要。
因为“自我演化”不是代码自己乱长，而是**规格、评测器、代码三者一起演化**。

你最值得抓住的一句话

A 窗口的反馈，最好不要直接变成代码；而是先变成“以后怎么判断对错”的规则。

比如你说：

- “这个回复太啰嗦”
- “这里应该先返回缓存，不要直接报错”
- “这个按钮放这里不顺手”
- “这个插件不应该访问这个目录”

这些都不只是一次性的 patch。
它们应该分别沉淀成：

- 风格 rubric
- 异常处理策略
- 交互规范
- 安全边界

然后系统再根据这些规则去改代码。

这样，下一次遇到类似问题，它不是“又来问你”，而是自己已经知道这算错。

所以，后台到底应该是什么

不是一个“万能 LLM”。
而是至少分成 4 个角色：

1. Spec/Judge 代理

负责把 A 窗口的话，变成机器能执行的东西：

- 需求说明
- 验收标准
- 失败定义
- 优先级
- 风险边界

2. Builder 代理

负责实现：

- 改代码
- 生成测试
- 跑 build / lint / test
- 给出变更说明

3. Observer 代理

负责盯 B 窗口和运行时：

- 收日志
- 看异常
- 收集用户操作轨迹
- 发现行为偏差

4. Deployer / Gatekeeper

负责上线和回滚：

- 是否满足自动发布条件
- 是否只发到本地
- 是否需要人工批准
- 回滚点在哪里

你真正要做的，不是“两窗口”，而是“两窗口 + 一个隐藏控制面”

更准确地说：

A窗口：产品控制面

它做两件事：

- 提新需求
- 给反馈、当裁判

B窗口：运行面

真实的软件就在这里被使用。
这里提供的不是“对话”，而是**证据**：

- 成功
- 失败
- 延迟
- 崩溃
- 误操作
- 用户不满意的痕迹

隐藏控制面：演化引擎

这个不一定给人看。
它在后台做：

- 将 A 的反馈结构化
- 将 B 的行为转成 incident
- 调用实现代理修复
- 跑 judge / test / replay
- 决定是否部署
- 更新系统记忆

为什么现在会显得笨重

因为现在这几个角色还是你在手工承担：

你现在实际上在做 5 件事：

- 你在说需求
- 你在判定哪里错
- 你在解释错因
- 你在做发布审批
- 你在记住“下次别再犯”

所以你才会觉得自己像坐在电脑旁边陪 AI 调试。

这不是因为思路错了。
而是因为**这些“隐性脑力”还没有被产品化**。

一旦产品化之后，你的人类工作会从“盯着它每次修”变成：

- 定义方向
- 给高价值反馈
- 只在高风险改动时审批

也就是说，人从“在线调试器”变成“系统裁判和产品 owner”。

用 Codex 来落这件事，位置其实很明确

Codex 适合放在**实现层和修复层**，而不是独自承担整套系统。
官方文档里几个点和你这个目标很贴合：

Codex 会在开始工作前读取 `AGENTS.md`；官方也建议把任务尽量明确成 `Goal / Context / Constraints / Done when`，把反复出现的规则沉淀进 `AGENTS.md`，这样它不是每次都靠临场猜。 OpenAI 开发者 +1

如果你想程序化控制本地执行器，可以用 Codex SDK；如果要把重复流程封装成可复用工作流，可以用 skills；如果要把日志、文档、issue、监控这些仓库外的东西接给它，用 MCP；如果要做后台定时任务，Codex app 里有 automations，但那条路要求 app 保持运行、项目在本地磁盘上。Codex CLI 还可以作为 MCP server，被上层 agent 编排。
OpenAI 开发者 +4

同时，Codex 默认并不是“无限权限自我进化器”：官方说明里，本地有 sandbox，网络默认关闭，并且有 approval policy 控制哪些操作必须停下来等你批准。所以它很适合当“受控执行器”，但你还是要在上层定义清楚边界和门禁。 OpenAI 开发者

这件事最容易做错的地方

很多人会把系统做成：

```
A窗口一句话 → LLM直接改代码 → 部署
```

这条链路短，但很不稳。
因为它缺了中间最关键的一层：**judge layer**。

更稳的做法应该是：

```
A窗口反馈
→ 先更新规格 / rubric / detector / eval
→ 再改代码
→ 再验证
→ 再部署
```

也就是：

先教系统“什么叫错”，再让系统去修。

真正的“自我演化”应该长什么样

你可以把它定义成一句话：

系统不是在自动写代码，而是在自动维护“意图—实现—验证”三者的一致性。

所以它演化的对象至少有 3 个：

1. `spec`
   软件应该做什么
2. `eval/judge`
   怎么判断它做得对不对
3. `code`
   实现当前 spec 的具体代码

这三样必须一起改。
只改 `code`，系统会反复犯同类错误。
只改 `spec`，系统不会落地。
只改 `judge`，系统会一直报错但不修。

对你来说，最小可运行版本应该是这样

先别做“完全自治的软件文明”，先做这个最小闭环：

第一步

A窗口输入需求，系统自动生成：

- `spec`
- `constraints`
- `acceptance criteria`
- 初始测试 / 验收脚本

第二步

Codex 实现并部署到本地 B 窗口。

第三步

B 窗口的运行日志、截图、用户操作、失败案例被后台收集。

第四步

后台判断两类问题：

- 明确异常：直接进入自动修复
- 主观偏差：汇报给 A 窗口，请你一句话裁定

第五步

一旦 A 给出反馈，系统不只是 patch，而是同时更新：

- `AGENTS.md`
- rubric
- detector
- regression test

这样同类问题就越来越少需要你亲自说。

最后把它压缩成一句定义

你要做的不是：

“A窗口告诉 AI 写什么，B窗口看看结果。”

你要做的是：

“A窗口定义意图和裁判标准，B窗口提供真实世界反馈，后台持续把反馈转成规则、测试和代码变更。”

这才叫自动反馈式的软件编写。

下一步最值钱的，不是继续聊抽象概念，而是把这套东西直接定成一个最小架构：
A窗口职责、B窗口职责、后台 4 个代理、以及 `spec` / `judge` / `code` 三类文件结构。

来源

如果我们给这个系统取个名字，应该是什么？ 你给几个备选，我来看下

Thought for 5m 39s

你这个系统的核心不是“写代码”，而是**意图 → 实现 → 使用 → 反馈 → 演化**。
所以名字最好带这几个气质：**闭环、反馈、演化、工厂感**。

我先给你一组比较像样的备选：

1. **回环 / HuiLoop**
   最贴你的模型。简短，中文感强，听起来就像“需求进来，结果回流”。
2. **意图回路 / IntentLoop**
   很准确，强调“A窗口说我要什么，系统把意图变成软件”。
3. **演化工坊 / EvoFoundry**
   平台感最强，像一个持续造软件、持续升级的软件工厂。
4. **反馈熔炉 / EchoForge**
   有“收到反馈就重铸实现”的感觉，偏硬核。
5. **反射工坊 / ReflexForge**
   强调“发现异常 → 自动反应 → 自动修复”，像神经反射弧。
6. **闭环引擎 / LoopEngine**
   偏底层、偏基础设施，适合做内核名或项目代号。
7. **生长引擎 / Growware**
   比较前卫，强调软件不是写完的，而是长出来的。
8. **回声工场 / EchoWorks**
   产品感更强，没那么技术宅，适合对外品牌。

我个人最看好的 3 个是：

回环：中文最好记，也最有你这个系统的味道。
IntentLoop：英文里最准确，技术人一看就懂。
EvoFoundry：最像一个“自动反馈式软件工厂”的正式产品名。

再给你一个判断标准：

- 想要**中文品牌感强**：选 **回环**、**演化工坊**
- 想要**技术味直接**：选 **IntentLoop**、**ReflexForge**
- 想要**像平台/基础设施**：选 **EvoFoundry**、**LoopEngine**

我不太建议直接叫“自愈系统”“自动修复引擎”这类名字，因为你的目标不只是修 bug，还是**自动造功能、自动演化软件**，名字太偏“修复”会把格局叫小了。

你挑一个方向后，我再顺着那个风格给你扩一版，更偏“硬核技术名”或者更偏“产品品牌名”。

auto-software 呢?

auto-software 可以，但更像**概念名**，不像**产品名**。

它的优点很明显：
一眼就懂，直接表达“软件自动生成、自动演化”。

但问题也很明显：
太泛、太直、太像类别词。看起来像一个方向、一篇文章标题，或者一个技术标签，不太像一个有记忆点的系统名字。
另外 `software` 太大了，你这套东西不只是“软件”，而是“**反馈驱动的软件演化系统**”。

我的判断是：

- 作为**总方向 / 理念名**：可以
- 作为**最终项目名 / 品牌名**：偏弱

更自然一点的做法是把它变成两层：

理念层叫： `Auto-Software`
系统名再单独取一个更有辨识度的名字

比如：

- **Auto-Software Loop**
- **Auto-Software Engine**
- **Auto-Software Foundry**
- **Auto-Software Lab**

这样就比单独的 `auto-software` 更完整。

如果你想保留这个味道，我会更推荐这几个变体：

- **AutoSoft**：短，但有点老派
- **SoftLoop**：更像闭环演化系统
- **AutoWare**：比 software 更像“可运行的产物”
- **EvoSoft**：更强调演化
- **SoftFoundry**：更像自动造软件的工厂

我的真实评价是：
auto-software 适合做你这个方向的“定义”，不太适合做最后的名字。

最像你的表达方式，其实是这种：

> **Auto-Software: a feedback-driven software evolution system**

也就是它做副标题很好，做主标题稍微弱了一点。

你要的话，我可以下一条直接给你分三种风格再起名：
极简技术风、产品品牌风、中文气质风。

Growware- grow software 呢？ 这两个比较呢

这两个里，**我更偏向** `Growware` 。

`Grow Software` 的优点是直白，几乎不用解释，别人一看就知道你在讲“让软件长出来”。但它更像一句描述，不太像名字。读起来像动宾短语，像 slogan，**不像一个系统名**。而且太通用，记忆点弱。

`Growware` 的问题是，第一次看的人可能会停半秒，想一下它是 `grow + software` 。但它的好处也正来自这里：**更像品牌名，更像一个独立概念**。它不是在说“做软件”，而是在说“会生长的软件系统”。

如果把两者放在一起比较：

Grow Software

- 清楚
- 但偏描述性
- 更像理念、标题、口号
- 不太像最终产品名

Growware

- 更有辨识度
- 更像平台 / 系统 / 引擎
- 更适合你这种“自动反馈、自动演化”的方向
- 但会稍微有一点点 `-ware` 的技术味，甚至会让少数人联想到 firmware / malware / adware 这种命名体系

所以关键看你要的气质：

你要的是**别人一眼懂**，那就偏 `Grow Software` 。
你要的是**像一个真的产品或系统名字**，那就偏 `Growware` 。

按你这个项目的目标，我会这样建议：

把 `Growware` **当主名**，把 `grow software` **当解释语**。

比如：

Growware
software that grows through feedback

或者：

Growware
grow software through use and feedback

这样就很顺了。
主名有辨识度，副句把理念讲清楚。

我个人排序会是：

1. Growware
2. Growware Loop
3. Growware Engine
4. Grow Software

如果你想，我下一条可以继续帮你把 **Growware** 往三个方向各打磨一版：更硬核、更产品化、或更有中文气质。

中文名字呢

已思考 39s ›

有，而且中文其实更容易做出“味道”。

如果按你这个系统的核心——**提出需求、自动实现、真实使用、接收反馈、后台演化**——我觉得中文名里最好带一点这三种感觉：**回路、成长、演化**。

我先给你一批比较像样的：

回环
我最推荐的一个。很干净，也最像“闭环系统”。
意思很贴你这个模型：需求进去，结果回来，继续演化。

迭生
这个我也很喜欢。
“迭代中生长”的感觉，短，硬一点，但不土。

自演
偏技术、偏内核名。
就是“自我演化”的压缩版，简洁，有点冷感。

生程
“生长”+“程序/工程”。
是造词，比较新，优点是有辨识度，缺点是第一次看要反应一下。

演化工坊
更像平台名。
听起来像一个持续造软件、持续改软件的地方。

回响工坊
更强调“反馈”。
用户一用，系统就有回响，再重塑实现。

意图回路
很准，尤其适合 A 窗口驱动的设定。
从“我要什么”开始，进入实现与反馈闭环。

生长引擎
稳妥、直接。
偏基础设施感，适合做系统内核名。

长成
这个比较中文化。
有“软件不是写出来的，是长成的”那种意味。

化程
“演化”+“程序”。
比较抽象，偏概念型，适合代号。

如果让我挑几档：

最适合做正式名字的
回环、迭生、演化工坊

最适合做技术内核名的
自演、生长引擎、意图回路

最适合承接 Growware 气质的
迭生、生程、长成

如果你问我现在最看好的 3 个，我会排：

1. 回环
最稳，最简洁，最像完整系统。

2. 迭生
很有“自动反馈式软件演化”的味道。

3. 自演
冷一点，但很适合做内核或引擎名。

如果你愿意，我下一条可以直接给你做一版：
偏品牌感、**偏技术感**、**偏中文意境感**，各出 5 个。

继续

可以，我按 3 条路线继续给你扩一版：**品牌感、技术感、中文意境感。**
我会尽量围绕你这个核心：**需求进入 → 自动实现 → 真实使用 → 自动反馈 → 持续演化。**

一、偏品牌感

这类名字适合以后真做成产品名，要求是：好记、顺口、有辨识度。

回环
还是最稳。短、好记、画面感强。
很适合你这个“闭环式软件生长系统”。

迭生
我很喜欢这个。
有“在迭代中生长”的意思，听着不像普通工具，更像一个新物种。

长成
很中文，也很有气质。
有一种“软件不是写出来的，是长成的”的味道。

生流
强调软件不是一次性交付，而是在流动中持续成形。
偏抽象一点，但挺现代。

回生
“反馈回来，再生长”。
比“回环”更柔一点，也更有生命感。

化成
有“意图化为成品”的意味。
偏文气，适合你这种既重技术又重表达的路子。

我在这组里最推荐的是：
回环、迭生、长成

二、偏技术感

这类更像系统名、引擎名、框架名，适合内部架构、开源项目、核心模块。

意图回路
最准。
直接对应你的 A 窗口：告诉系统我要什么，然后进入实现与反馈闭环。

生长引擎
非常稳。
听起来就像底层系统，不花，但清楚。

自演引擎
“自我演化引擎”的压缩。
有点冷，但很像内核名。

演化回路
比“意图回路”更强调持续修正与自动更新。
适合做整体系统名。

反馈内核
强调你的系统不是单次生成，而是靠反馈驱动。
很适合技术文档里的核心模块名称。

自举工厂
带一点系统味。
有“自己推动自己继续完善”的感觉。

演化栈
适合偏工程系统的感觉。
像一个完整 software loop 的 infrastructure stack。

我在这组里最推荐的是：
意图回路、生长引擎、演化回路

三、偏中文意境感

这类更有气质，适合你想让名字不那么工具味，而更像一个理念系统。

化生
很简短。
“化”是变化、转化；“生”是生长、生成。
很贴你这个系统：把意图化成软件，让软件继续生长。

回响
强调使用之后，系统有回应、有修正。
比较柔，但很美。

渐成
一点一点长出来、修出来。
适合表达“软件不是一次完成，而是持续成形”。

流成
在流动中成形。
有点哲学感，适合你前面一直在想的“软件自动演化”。

再生
每次反馈都是一次重构和新生。
偏大气，但稍微泛一点。

成环
“形成闭环”的压缩感。
简洁，挺适合作为概念名。

生界
像一个会长出软件的系统空间。
有些抽象，但记忆点不错。

我在这组里最推荐的是：
化生、回响、渐成

四、如果和 Growware 对应，中文最贴近的几种

如果你心里还是偏爱 **Growware** 那种“会生长的软件”气质，那中文最贴的不是“自动软件”，而是这几个：

迭生
最像。
既有 grow，又有 evolve。

长成
最自然。
很像“grow software”的中文表达。

化生
更抽象、更有系统感。
像“由意图化为软件，并继续生长”。

生长引擎
最工程化。
适合做技术名。

五、如果要我现在直接帮你收敛

我会给你三档建议：

最适合做正式项目名

回环
稳、清楚、记忆点够。

最适合做新概念名

迭生
最有“自动反馈式软件演化”的味道。

最适合做底层系统 / 引擎名

生长引擎
不花，落地，技术人一眼懂。

六、一个实用组合方式

你甚至可以用“双层命名”：

回环 · 生长引擎
迭生 · 演化回路
Growware / 迭生
回环（Feedback-Driven Software Loop）

这样既有中文主名，也有英文解释，气质会完整很多。

我个人此刻最看好的 5 个排序是：

回环
迭生
长成
化生
生长引擎

你要是愿意，我下一条可以继续只围绕这 5 个，帮你做一版：
含义、气质、适合当品牌名还是系统名、以及 slogan。

生长引擎

生长引擎，这个名字是成立的，而且挺好。

它的优点在于两层意思都很准：

“**生长**”不是一次性写完，而是根据使用和反馈继续长出来。这个词很贴你要的系统本质。
“**引擎**”说明它不是一个小功能，而是底层驱动器、演化核心、持续运转的机制。

所以这个名字最适合的定位是：

一个自动反馈式软件系统的核心引擎名。
也就是说，它特别像“内核名”“系统名”“技术底座名”，很稳，很像真的能落地的东西。

我觉得它比“自动软件”这类名字更好，因为它不是在说结果，而是在说**机制**。
你的目标也确实不是“自动生成软件”这么简单，而是“让软件在反馈里持续生长”。
“生长引擎”把这个意思抓住了。

它的小缺点也有一个：
这个名字**偏技术、偏基础设施**，品牌感没有那么轻盈。
所以如果以后你想对外做成消费级产品名，它会显得稍微硬一点；
但如果你是拿它做项目主名、架构名、开源系统名，其实很合适。

我会这样理解它：

生长引擎 = 根据需求、使用、反馈，持续驱动软件演化的核心系统

如果你决定用这个名字，下面这几句都可以直接配套：

生长引擎
让软件在反馈中持续长成

生长引擎
让需求自己长成可用的软件

生长引擎
从意图出发，在使用中演化

生长引擎
不是写完软件，而是养成软件

如果再往前走一步，我会建议你这样定：

项目总名就叫 **生长引擎**
英文副名可以配成：**Growth Engine for Software**
或者更有一点系统味：**Software Growth Engine**

我个人判断：
生长引擎 这个名字，已经可以进入正式候选了。
它不花，但有方向感，也有你这套系统真正的灵魂。

要是继续往下定，我建议下一步就做两件事：
先把它的一句话定义和一句 slogan 定死。

生长引擎 = 根据需求、使用、反馈，持续驱动软件演化的核心系统
这个解释和 a feedback-driven software evolution system； grow software through use and feedback 比较匹配；

对，**是匹配的**，而且匹配度还挺高。

可以这样理解：

生长引擎 = 名字
根据需求、使用、反馈，持续驱动软件演化的核心系统 = 中文定义
a feedback-driven software evolution system = 英文定义
grow software through use and feedback = 英文 slogan / 副标语

这里面有一个细微差别：

- **“a feedback-driven software evolution system”** 更像**正式定义**
- **“grow software through use and feedback”** 更像**一句品牌表达**

所以你这套搭配其实很顺：

生长引擎
根据需求、使用、反馈，持续驱动软件演化的核心系统。
A feedback-driven software evolution system.
Grow software through use and feedback.

不过你中文里有“**需求**”，而英文这两句里对“需求”体现得还不够强。
为了更严丝合缝，可以把英文定义再收一下：

生长引擎
根据需求、使用与反馈，持续驱动软件演化的核心系统。
A feedback-driven engine that continuously evolves software from intent, use, and feedback.

这样就更完整了。
其中：

- **intent** 对应“需求 / 意图”
- **use** 对应“使用”
- **feedback** 对应“反馈”
- **evolves software** 对应“驱动软件演化”

我建议你最后定成这种三层结构：

主名： 生长引擎
定义： 根据需求、使用与反馈，持续驱动软件演化的核心系统。
英文： A feedback-driven engine that continuously evolves software from intent, use, and feedback.
标语： Grow software through use and feedback.

这已经很像一个正式项目的命名体系了。